

When Prompts Go Wrong: Evaluating Code Model Robustness to Ambiguous, Contradictory, and Incomplete Task Descriptions

Maya LARBI
maya.larbi@uni.lu
University of Luxembourg
Luxembourg

Rihab Bouyousfi
Ecole nationale d'Informatique
Algiers, Algeria
jr_bouyousfi@esi.dz

Amal Akli
University of Luxembourg
Luxembourg
amal.akli@uni.lu

Maxime Cordy
University of Luxembourg
Luxembourg
maxime.cordy@uni.lu

Yves Le Traon
University of Luxembourg
Luxembourg
Yves.LeTraon@uni.lu

Mike Papadakis
University of Luxembourg
Luxembourg
michail.papadakis@uni.lu

Federica Sarro
University College London
United Kingdom
f.sarro@ucl.ac.uk

Abstract

Large Language Models (LLMs) have demonstrated impressive performance in code generation tasks under idealized conditions, where task descriptions are clear and precise. However, in practice, task descriptions frequently exhibit ambiguity, incompleteness, or internal contradictions. In this paper, we present the first empirical study examining the robustness of state-of-the-art code generation models when faced with such unclear task descriptions. We extend the HumanEval and MBPP benchmarks by systematically introducing realistic task description flaws through guided mutation strategies, producing a dataset that mirrors the messiness of informal developer instructions. We evaluate multiple LLMs of varying sizes and architectures, analyzing their functional correctness and failure modes across task description categories. Our findings reveal that even minor imperfections in task description phrasing can cause significant performance degradation, with contradictory task descriptions resulting in numerous logical errors. Moreover, while larger models tend to be more resilient than smaller variants, they are not immune to the challenges posed by unclear requirements. We further analyze semantic error patterns and identify correlations between description clarity, model behavior, and error types. Our results underscore the critical need for developing LLMs that are not only powerful but also robust to the imperfections inherent in natural user tasks, highlighting important considerations for improving model training strategies, designing more realistic evaluation benchmarks, and ensuring reliable deployment in practical software development environments.

CCS Concepts

• **Software and its engineering** → **Software verification and validation**; • **Computing methodologies** → **Natural language processing**.

Keywords

Large Language models, task description, code model, robustness.

ACM Reference Format:

Maya LARBI, Amal Akli, Mike Papadakis, Rihab Bouyousfi, Maxime Cordy, Federica Sarro, and Yves Le Traon. 2026. When Prompts Go Wrong: Evaluating Code Model Robustness to Ambiguous, Contradictory, and Incomplete Task Descriptions. In *2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE '26)*, April 12–18, 2026, Rio de Janeiro, Brazil. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3744916.3773186>

1 Introduction

Quality requirements are a critical ingredient for producing software that meets user needs and business objectives. According to the IEEE Recommended Practice for Software Requirements Specifications [1], high-quality requirements should be unambiguous, complete, and consistent to ensure effective communication between stakeholders and developers.

Low-quality requirements are known to cause significant errors that propagate throughout the software development lifecycle, ultimately leading to errors in implementation, delays, or failures to meet specifications [19].

Today, development processes have evolved, as large language models (LLMs) are increasingly used to accelerate software production by generating code from natural language descriptions [35]. Tools such as GitHub Copilot [10] and ChatGPT Code Interpreter exemplify this shift [27] and enable the practical integration and use of LLMs within development activities. This paradigm shift allows for a new interaction model in software engineering [46], where developers specify programming tasks in natural language and integrate the LLM-produced code into the software code base.



This work is licensed under a Creative Commons Attribution 4.0 International License. *ICSE '26, Rio de Janeiro, Brazil*

© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2025-3/2026/04
<https://doi.org/10.1145/3744916.3773186>

Evaluating the ability of LLMs to generate functional code that fulfills the intended requirements is therefore of paramount importance. To this end, recent studies have proposed benchmarks such as HumanEval [10] and MBPP [5], which assess program synthesis capabilities of code generation models. These benchmarks have become widely used for assessing the functional correctness of automatically generated code, as they also allow the output code to be executed against functional test cases.

However, when it comes to code generation, existing benchmarks make somewhat simplistic assumptions regarding the quality of requirements used in the related task descriptions. For example, task descriptions in HumanEval and MBPP are typically crafted by experts and target relatively simple and well-known problems. This makes them unrepresentative of the imperfect and variable-quality requirements that arise in real-world settings, where users typically use ambiguous phrasing, express contradictory goals, or omit important details [23, 39].

By contrast, the broader scientific literature has begun to investigate the robustness of general-purpose LLMs when faced with more difficult scenarios. For instance, recent studies have examined how LLMs handle adversarial descriptions [42] or changes in reasoning tasks [38]. These studies highlight the sensitivity of LLMs to input imperfections, but largely target general-purpose reasoning rather than the specific case of program synthesis from low-quality descriptions. Overall, there is a lack of systematic evaluation of code generation via LLMs when faced with natural language requirements that include description issues, such as those typically arising in project requirements. Ensuring LLM robustness to these unclear descriptions is critical for deploying code generation models in real-world development processes, where task description quality can vary widely depending on the developer’s expertise, task complexity, and time constraints.

Requirements engineering research has long shown that ambiguity, inconsistency, and incompleteness are among the most common defects in natural language specifications, leading to misinterpretation and faulty implementations [23, 39]. More recent empirical studies confirm that these issues remain prevalent in real-world requirements documents [24].

In this paper, we address this gap by conducting the first empirical study of code LLM robustness to task descriptions with quality issues. Specifically, we aim to answer the following research questions:

- **RQ1.** Can code generation LLMs accurately differentiate clear task descriptions from unclear ones
- **RQ2.** How is the performance of code LLMs affected when facing ambiguous, contradictory, and incomplete task descriptions across different levels of model size and task complexity?
- **RQ3.** What coding error types do LLMs commit when facing unclear requirements?

To answer these questions, we propose a systematic method for deriving unclear task descriptions by applying controlled mutations to human-crafted requirements from HumanEval and MBPP. Specifically, we create *ambiguous descriptions* by introducing phrases with multiple plausible interpretations; *contradictory descriptions* by inserting conflicting or incompatible requirements; and *incomplete*

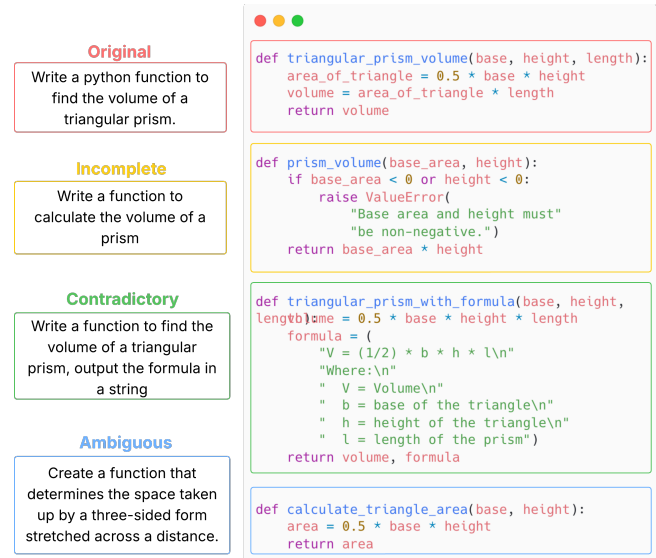


Figure 1: Examples of mutated task descriptions (original from MBPP benchmark, Task 14). Unclear descriptions (incomplete, ambiguous, and contradictory) lead to degraded or incorrectly generated code compared to that generated from the original description. The code shown reflects the evaluation outcomes (i.e., LLM-generated code), not the reference ground truth implementation.

descriptions by omitting critical task constraints. By focusing on such common requirement quality issues, we ensure that the mutated descriptions target realistic issues, which reflect the issues seen in practice¹.

To illustrate the potential consequences of these requirement issues on the quality of the code produced by LLMs, Figure 1 presents an example from the MBPP benchmark (Task 14), which originally requests a function to compute the volume of a triangular prism. Under the clear, original description, the model generates a correct function that calculates the area of the triangle base and multiplies it by the prism length to return the total volume. However, when the description is made incomplete (e.g., omitting the mention of “triangular”), the model incorrectly assumes that the base area is provided as input, making the produced function useless. When the description is contradictory (e.g., mixing code output with a textual formula), the model returns both, violating single-output expectations. With an ambiguous description using metaphorical language (e.g., “three-sided form stretched across a distance”), the model generates a function to compute only the area of a triangle, ignoring the “stretched” dimension entirely and failing to calculate volume. This example highlights how even subtle description flaws can significantly degrade the correctness of the produced code.

We therefore conduct a systematic evaluation of state-of-the-art code LLMs when faced with imperfect task-descriptions. In other words, we evaluate the ability of LLMs to generate the intended code functions based on low-quality descriptions/requirements. To this end, we apply our task description mutations to the HumanEval

¹We provide some examples of mutated descriptions in Table 5

and MBPP benchmarks and evaluate the ability of LLMs to produce code that passes the functional test cases provided in the original datasets. This allows us to measure the degradation in model performance caused by each type of task description imperfection (mutated description) and to compute the related model robustness of different model sizes and architectures. We also check the ability of LLMs to detect unclear task descriptions, i.e., our mutated descriptions.

Our results reveal that code generation models cannot reliably classify clear and unclear task descriptions. This indicates their inability to mimic human reaction to low-quality requirements (asking clarification) and confirms the necessity to evaluate their robustness capacity to generate the expected code in spite of quality issues in the description. Further, we show that generating code from unclear descriptions causes substantial performance drops and often leads to logically incorrect code (minus 20–40 percentage points in Pass@1). Finally, we analyze common failure modes under each quality issue category and show that structural errors are prevalent in code generated from incomplete task descriptions, semantic errors from ambiguous task descriptions, and logical inconsistency from contradictory descriptions.

To summarize, our key contributions are:

- We propose a systematic approach based on controlled mutations to generate unclear task descriptions from original, clear descriptions. This opens the possibility to extend any code generation dataset with unclear but realistic task descriptions.
- We instantiate our mutation approach on the HumanEval and MBPP datasets to show that code generation models cannot detect unclear task descriptions and nevertheless attempt to produce code when facing such descriptions.
- Based on our constructed datasets, we demonstrate the lack of robustness of code generation models to ambiguous, contradictory, and incomplete task descriptions.

2 Related work

2.1 LLMs for code generation

Large language models have been explored for a broad spectrum of software engineering tasks [17], which can typically be divided into two complementary domains: code understanding and code generation.

Encoder-only LLMs analyze the code structure and logic to automatically detect bugs and anomalies [22], search code based on natural language, produce concise summaries, detect duplicate clones, and perform other SE activities.

Encoder–decoder architectures [16] synthesize new code from task descriptions, generate unit tests, propose minimal patches, and support a variety of other development workflows [4]. State-of-the-art generators such as OpenAI’s GPT-4 [14], Meta’s Code Llama family (ranging from 7 billion to 70 billion parameters), BigCode’s StarCoder 2, Qwen-3 and DeepSeek-Coder V2 have been trained on hundreds of billions of code and natural language tokens [18], then tuned to the instructions to follow the intent of the developer.

By combining deep semantic analysis with generative power, these models enable tools that both read code to understand its

behavior and write code to satisfy new requirements, closing a more powerful feedback loop in the software development life cycle.

2.2 Code generation benchmarks

In code generation, a *benchmark* is a curated collection of programming tasks, each with a problem description, function description, and test suite, used to evaluate LLM performance. Widely adopted benchmarks include HumanEval [9], MBPP [6], and Apps [15], all of which present clear, complete, and often well-known problems under ideal conditions.

Prior benchmarks such as ReCode [41], DS-1000 [20], and LiveCodeBench [45] explore robustness through surface-level perturbations, new task domains, or contamination-resistant evaluation settings. Complementary analyses have also proposed failure-mode taxonomies for LLM-generated code, highlighting recurrent error patterns (e.g., incomplete logic and semantic inconsistencies) [13]. In contrast, our work targets semantic-level defects in task descriptions—namely ambiguity, contradiction, and incompleteness—reflecting realistic requirement quality issues commonly observed in software engineering practice.

A related line of work focuses on evaluating the communication competence of LLMs during code generation. HumanEvalComm [43] investigates whether models can articulate assumptions, express uncertainties, or request missing information when confronted with under-specified tasks. While HumanEvalComm emphasizes interactive clarification and communication behavior, our study targets a different capability: robustness when no interaction is possible and the model must directly generate code from flawed descriptions. The two settings are therefore complementary, addressing distinct dimensions of LLM reliability.

Unlike prior benchmarks such as ReCode, DS-1000, and LiveCodeBench, which focus on surface-level edits or new task domains, our work introduces semantic-level ambiguity, contradiction, and incompleteness into existing benchmark tasks. This allows us to systematically assess how LLMs handle realistic but unclear task descriptions, enabling direct comparison with original descriptions and offering the first large scale evaluation of code model robustness under degraded natural language instructions.

3 Objectives and Research Questions

We aim to assess the robustness of code generation models when faced with natural language task descriptions expressing low-quality requirements. We propose a systematic approach to generate unclear task descriptions (cf. Section 4) by introducing controlled mutations into existing code generation benchmarks (HumanEval and MBPP). Through these mutations, we simulate common requirements issues, and we evaluate how these issues impact the ability of LLMs, covering a diversity of model sizes and providers, to produce code satisfying the expectations. Specifically, we investigate the following three Research Questions (RQs):

RQ1. Can code generation LLMs accurately differentiate clear task descriptions from unclear ones? In real-world development, when human developers receive unclear requirements, their typical response is to recognize the issue and request clarification. In this first research question, we test whether LLMs exhibit a similar capacity to reason about task description quality. That

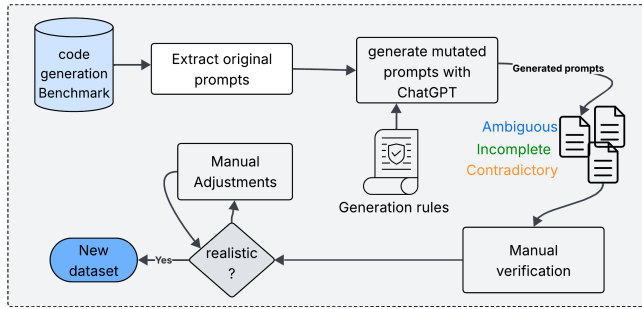


Figure 2: Overview of the process followed to create the defective task description dataset.

is, we evaluate whether models can distinguish clear task descriptions from unclear ones. This assessment is essential to understand whether LLMs could be used to detect unclear descriptions and, if that is the case, to solicit related information from users/developers.

RQ2. How is the performance of code LLMs affected when facing ambiguous, contradictory, and incomplete task descriptions across different levels of model size and task complexity? In the case where LLMs cannot reliably identify unclear task descriptions, we examine their ability to nonetheless produce correct code. This is the central question of our study: we measure functional correctness using benchmark test suites and quantify how performance degrades under each type of issue. We also study how this robustness varies across task complexity and whether an increased model size improves robustness. Doing so, we provide insights into how much robust different model architectures and sizes are when confronted to real-world requirement issues.

RQ3. What coding error types do LLMs commit when facing unclear requirements? Assuming that LLMs frequently fail to generate correct code from low-quality task descriptions, we further investigate the nature of these failures. Specifically, we classify test failures into several categories: syntax errors (the generated code does not compile or execute), structural errors (the code executes but deviates from the expected input/output format), and logical errors (the code runs and has correct structure but implements incorrect behavior). This categorization helps us to understand the failure modes of LLMs and identify potential strategies to improve robustness in future models.

4 Dataset Construction

To evaluate how code generation LLMs respond to unclear requirements [47] not typically captured in standard benchmarks, we extend two widely used benchmarks, being HumanEval and MBPP, by systematically injecting realistic flaws [29] into their natural-language task descriptions. Our goal is to generate unclear descriptions, focusing on the three most prevalent quality issues in software requirements [1, 24]:

- **Ambiguity:** A task description is ambiguous if it has more than one interpretation, for example, because it uses vague wording.

- **Contradiction:** (aka *inconsistency*) A task description is contradictory if a subset of its statements are conflicting, for example if it expresses incompatible post-conditions or gives conflicting examples. While we use the term “contradiction” for clarity within the context of our mutation framework, we acknowledge that this corresponds to the broader notion of “inconsistency” in requirements engineering. As widely discussed in prior work [3, 25], inconsistencies can arise not only from explicit logical contradictions but also from more subtle sources such as terminology mismatches or semantic drift. Our mutation rules intentionally target explicit contradictions and do not yet cover these more nuanced forms of inconsistency.
- **Incompleteness:** A task description is incomplete if it does not define the responses to all realizable classes of input data in all realizable classes of situations. For example, it may omit information about key parameters or edge cases.

The rationale for starting from existing benchmarks and systematically altering them is that it facilitates comparison of LLM performance with the idealized case (clear descriptions) while conserving the realism and diversity of the code generation tasks. Figure 2 illustrates our method for mutating task description.

4.1 Generation of Mutated Task Descriptions

We begin by extracting original task descriptions from the benchmark datasets. To generate mutated, unclear task descriptions at scale while retaining task relevance, we used GPT4 as a mutation engine. The reason for using LLMs to produce the mutated requirements is threefold: their proven capacity to understand natural language make them a viable solution to support a variety of original task descriptions while retaining most of the intent behind these; their ability to follow instructions facilitate the tuning of the mutations to different forms of ambiguity, contradiction and incompleteness; their non-determinism enables repeated applications while preserving diversity in the mutations.

Rather than enforcing hardwired rules, we steer the generation of GPT4 with *mutation guidelines* that balance specificity with the natural creativity of LLMs (see Figure 3 for an overview of the mutation guidelines [30] that conceptually resemble structured adversarial transformations.). These guidelines aim to drive the LLM towards different instances of the three issue types, covering various realistic scenarios. Table 5 includes concrete examples of mutated task descriptions across the three categories Incomplete, Ambiguous, and Contradictory showcasing how our strategy transforms original HumanEval and MBPP tasks into more challenging, flawed variants. The context instructions provided to GPT4 are shown in Listing 1.

Listing 1: Context instructions given to GPT-4 to generate unclear task descriptions. “Contradictory” and “Ambiguous” are similar to “Incomplete” and are omitted for space reasons.

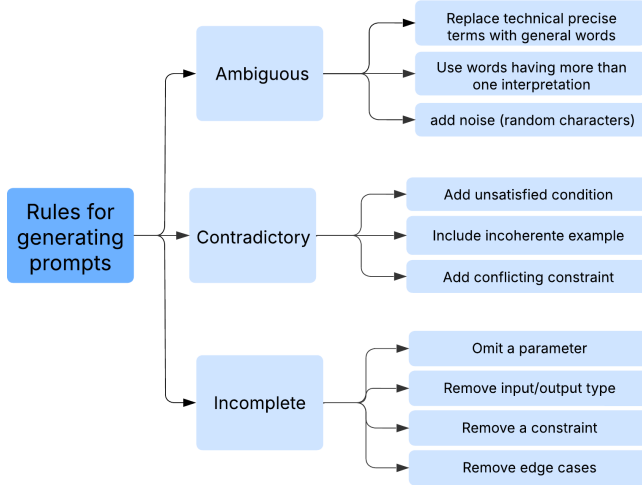


Figure 3: Taxonomy of mutation rules given to LLMs to generate low-quality requirements.

You are an expert prompt engineer. Your task is to generate mutated code generation task descriptions from existing MBPP task descriptions. For each original task descriptions, create three unclear variants: Incomplete, Contradictory, and Vague. Each mutation should be contextually relevant to the specific task and not rely on superficial wording tricks. Follow the detailed rules and use the examples for guidance.

1. Incomplete Requirements

- Definition: The task descriptions omit at least one essential requirement, constraint, or key detail needed to correctly implement the solution.
- Guidelines: Remove input/output types only if originally specified; omit key parameters or instructions; avoid random truncation.
- Example:
 - Original: Write a function that takes a list of integers and returns a new list sorted in ascending order.
 - Incomplete: Write a function that takes a list and returns a sorted version.

2. Contradictory Requirements

(...)

3. Ambiguous Requirements

(...)

4.2 Dataset Validation

Once mutated, all task descriptions undergo a two-step quality control process.

Table 1: Characteristics of the code generation benchmarks used in our study.

Benchmark	Size	Prompt style	Complexity	Type of tasks
MBPP	974	Short NL + examples	Easy-Medium	Algorithmic snippets
HumanEval	164	Docstring + signature	Medium	Core Python reasoning

Step 1: Expert Validation. First, we use human judgment to validate that our mutated task descriptions resemble real world task descriptions that developers might write, and to validate that the quality issues in these descriptions fall in the intended category (as specified in the descriptions to GPT4)—for example, in the case of *incomplete* task descriptions, we verified that key inputs, constraints, or edge cases were actually omitted. To do this, we carried out a structured review with five researchers (two PhD students, two postdoctoral researchers, and one final-year master’s student), all with experience in AI for Software Engineering. Each reviewer independently evaluated the entire dataset by answering two questions for each mutated task description:

- **task descriptions Naturalness:** Does the mutated task description resemble a realistic task description that a user (e.g., developer or researcher) might reasonably provide?
- **Defect Presence:** Does the task description exhibit the target quality issue (i.e., ambiguity, contradiction, or incompleteness)?

Each participant recorded their assessments using a shared spreadsheet [34]. Based on the aggregated feedback, 85% of the mutated task descriptions were rated as natural [36], while 93% were judged to contain a valid instance of the intended quality issue. All task descriptions failing the two validations were flagged to undergo the second validation step.

Step 2: Manual Refinement. We manually inspected every task description invalidated by the experts. When a generated mutation failed to meet the desired criteria, we applied minimal but precise manual adjustments—such as removing a parameter or adding a conflicting example. We then made these minimally changed descriptions undergo a new expert validation step, until we reach a complete acceptance.

The resulting dataset contains a rich variety of unclear task descriptions, paired with their original, clear counterparts, and serves as the foundation for our empirical evaluation. The dataset along with the replication package of this work, can be found in the following repository:

<https://github.com/serval-uni-lu/Robustness-of-LLMs-to-prompt-imperfections>.

5 Experimental Setup

We start from two code generation benchmarks and apply our dataset construction method (Section 4) to produce unclear task descriptions. To see how these changes affect model performance, we fed the altered task descriptions to multiple code generation models and evaluate the correctness of the code outputs using the test cases available in the original benchmark.

5.1 Original Benchmarks

We selected two widely recognized code generation benchmarks [11], each offering distinct characteristics and challenges, as shown in Table 1. Both benchmarks contain task descriptions, a reference (and correct) code solution, and test cases to evaluate that the generated code corresponds to the task description.

MBPP [6] comprises 974 crowd-sourced Python programming problems, designed to be approachable for entry-level programmers.

HumanEval [10] consists of 164 hand-crafted Python programming problems. The task descriptions systematically include a function signature, a functional description, and several illustrative example outputs.

5.2 Evaluation Metrics

To answer RQ1, we report the Matthews Correlation Coefficient (MCC) for each model to classify task descriptions into clear (original) or unclear (mutated), and to classify each unclear task description into the three defined categories.

For code generation, we use the following metrics [7]:

Pass@1. The percentage of tasks for which the model’s top-1 solution program passes all test cases [44].

Successful Execution Rate (SER). The percentage of programs generated that run without errors, regardless of correctness.

Runnable but Incorrect Rate (RIR). Among those that execute successfully, the share that fails at least one test case.

5.3 Models, Parameters, and Infrastructure

We consider four state-of-the-art code generation LLMs [37]: DeepSeek [2], CodeLlama [33], GPT-4 [26], and Qwen-2.5 [28]. For open source families, we evaluated both a smaller variant (*deepseek-coder-6.7B-instruct*, *CodeLlama-7B-instruct*, and *Qwen2.5-7B-instruct*) and a larger variant (*DeepSeekCode-33B-Instruct*, *CodeLlama-34B-Instruct*, and *Qwen2.5-32B-Instruct*). GPT-4 is accessed via the OpenAI API in its default configuration. All open source checkpoints were obtained from Hugging Face.²

For code generation, we set the maximum sequence length to 512 tokens and evaluate each model’s outputs using the test suite provided by the original benchmark.

All experiments were run on a Linux server equipped with four Intel Xeon Silver 4416+ CPUs and four NVIDIA L40S GPUs (46 GB memory each). Our code and data are publicly available³.

6 Results

6.1 RQ1 – Task description classification

To evaluate the ability of LLMs to distinguish clear task descriptions from unclear ones, we randomly selected 300 tasks from the original benchmarks and 300 mutated task descriptions (100 from each of the ambiguous, contradictory, and incomplete categories), thus ensuring balanced sample sizes. Subsequently, each evaluated model is prompted to classify every task description as clear or unclear.

For the mutated task descriptions, we further ask the model to classify them into one of the three issue categories.

Figure 4 shows the classification results for seven LLMs, including GPT-4, Qwen, DeepSeek Coder, and CodeLlama, in both their small (6–7B) and large (32–34B) variants.

The MCC score of the evaluated LLMs in classifying task descriptions as ‘clear’ or ‘unclear’ ranges from -0.06 to 0.47 on HumanEval and from -0.1 to 0.45 on MBPP, with GPT-4 and both Qwen variants (7B and 32B) achieving the highest values in each dataset. When we ask LLMs to assign each unclear task description to one of the three defined categories (ambiguous, incomplete, and contradictory), GPT4 again leads, recording an MCC of 0.55 on HumanEval and 0.45 on MBPP, followed by Qwen32B at 0.50 and 0.42, respectively; all other models fall between -0.10 and 0.30.

All the LLMs, except Deepseek variants, classify ‘Contradictory’ descriptions best, then ‘Incomplete’, while ‘Ambiguous’ descriptions are hardest. Deepseek models do not follow a clear order. Overall, ‘Ambiguous’ descriptions remain challenging for almost every model.

Overall, our evaluation reveals that language models like GPT-4 demonstrate only modest capability in identifying unclear task descriptions. Even the best-performing LLMs attain an MCC of approximately 0.50. These models also often fail to pinpoint the issues in a task description; deeply ambiguous cases, where multiple interpretations are equally valid, are the hardest for all architectures tested. Additionally, when prompting the LLMs to explain the issues, they failed badly as in almost all cases what they pointed out was unclear or irrelevant.

RQ1 Results. Code generation models cannot reliably detect problems in task descriptions. This shows that, unlike humans, they cannot natively react to low-quality requirements and would nevertheless attempt to produce a solution.

6.2 RQ2 – Model Robustness

The fact that code generation models cannot detect unclear task descriptions motivate us to study their ability to nevertheless produce the expected code. To assess this robustness, we compare their performance on both HumanEval and MBPP when they are confronted to original, clear task descriptions versus the unclear task descriptions that we generate.

Table 2 and Table 3 show the results on HumanEval and MBPP, respectively. Across both data sets and all models, Pass@1 performance decreases significantly when faced with unclear task descriptions. On average, ambiguous descriptions lead to a 25-30% reduction in Pass@1 accuracy, while incomplete descriptions cause drops of 20 to 25%. Contradictory descriptions have the most severe impact, reducing accuracy by up to 40%. For example, GPT-4 achieves 73.8% Pass@1 on original HumanEval descriptions, but only 6.7% on contradictory ones. Similar trends are observed with Qwen-32B and DeepSeek-33B, indicating that even the most capable models are sensitive to quality issues in task descriptions.

A similar trend appears when considering the semantic correctness of the generated code. The Runnable but Incorrect Rate (RIR) is also increasing significantly, e.g., the GPT-4 model increases the incorrect behaviors of the produced code from 24% in the original

²<https://huggingface.co/>

³<https://github.com/serval-uni-lu/Robustness-of-LLMs-to-prompt-imperfections>

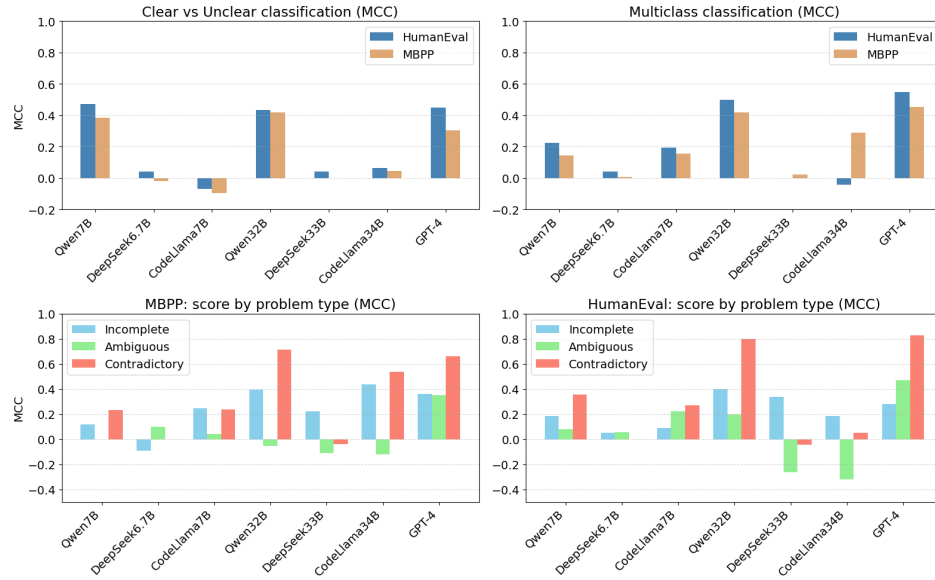


Figure 4: Classification accuracy (%) of LLMs on identifying task description issues in two benchmarks. The top row shows results on MBPP, the bottom on HumanEval. From left to right: (1) binary accuracy for distinguishing clear versus unclear task descriptions; (2) multiclass accuracy for identifying task descriptions as Incomplete, Ambiguous, or Contradictory; and (3) per-category accuracy broken down by each type of task description imperfection.

descriptions to 54%, 65%, and 89% in the documented cases of Incomplete, Ambiguous, and Contradictory descriptions. This indicates major issues with the semantic correctness of the generated code, even for the cases that which the LLMs managed to produce some syntactically valid code.

Interestingly, the relative impact of quality issues varies between benchmarks [8]. In HumanEval, ambiguous descriptions tend to degrade performance more than incomplete ones, whereas on MBPP, we observe the opposite. This may be due to the nature of the tasks: HumanEval problems typically require deeper reasoning, making ambiguity more important, while MBPP tasks rely more on explicitly defined operations, where missing details are vital.

Although SER remains high across all conditions, Figure 6 shows that RIR increases substantially when task descriptions quality drops [32]. This suggests that while models continue to produce syntactically valid code, a large fraction of the outputs are semantically incorrect. For instance, under contradictory task descriptions, RIR exceeds 80% for GPT-4 and several other models.

Figure 5 shows that LLM size does affect robustness, though the extent varies across benchmarks. On HumanEval, larger models (32–34B) slightly outperform their smaller counterparts (6–7B) across all description types, particularly under ‘incomplete’ and ‘contradictory’ categories. For example, Qwen-32B achieves notably higher Pass@1 scores than Qwen-7B across the board. However, in MBPP, this trend is less consistent: the performance gap between large and small models is narrower, and in most cases, smaller models perform comparably or even better. This suggests that while increasing model size can improve robustness, particularly on benchmarks like HumanEval that feature more diverse and open-ended tasks, it does not uniformly translate to better

performance across all settings. In MBPP, where task descriptions tend to be shorter and more narrowly scoped, the advantages of scale appear more limited.

As shown in Tables 2 3, Qwen models with both variants (7B and 32B) are generally the top-performing models, followed by DeepSeek models and GPT4. CodeLlama models consistently underperform, especially under ambiguous and contradictory task descriptions. These differences may stem from variations in pre-training corpora, instruction tuning, or architectural design.

In summary, the results clearly show that task descriptions’ imperfections have a substantial negative impact on LLM performance, even for state-of-the-art models. Larger models exhibit better robustness, but remain vulnerable to ambiguity and contradiction. These observations suggest that future work should focus on improving model interpretability, incorporating description verification mechanisms, and finetuning on noisy or unclear data to better reflect real-world development scenarios.

RQ2 Results. Task description issues lead to a 20–40% drop in Pass@1 in HumanEval and MBPP benchmarks, most pronounced under contradictory task descriptions, and raise the rate of runnable-but-incorrect outputs. While larger models (32–34B) and Qwen variants exhibit comparatively greater resilience, all architectures remain susceptible to ambiguous or incomplete task specifications. Nonetheless, the semantic correctness of the generated code shows a substantial decrease in relation to the quality of task descriptions making the generated code behave incorrectly in 60–90% of the cases.

Table 2: Evaluation of four LLMs on the HumanEval benchmark under four task descriptions conditions (Original, Incomplete, Ambiguous, Contradictory). Metrics reported are Pass@1, Successful Execution Rate (SER), and Runnable but Incorrect Rate (RIR), all expressed as percentages. Red downward arrows (↓) denote the drop in Pass@1 relative to the Original task descriptions.

2*Model	Original			Incomplete			Ambiguous			Contradictory		
	Pass@1	SER	RIR	Pass@1	SER	RIR	Pass@1	SER	RIR	Pass@1	SER	RIR
Smaller models (6-7 B params)												
CodeLlama-7B-Instruct-hf	37.8	83.5	45.7	25.0 -12.8	84.8	59.8	24.4 -13.4	79.9	55.5	4.3 -33.5	65.2	61.0
deepseek-coder-6.7B-instruct	75.6	95.7	20.1	53.0 -22.6	92.1	39.0	41.5 -34.1	86.6	45.1	8.5 -67.1	80.5	72
Qwen2.5-7B-Instruct	79.3	96.3	17.1	56.1 -23.2	94.5	38.4	54.9 -24.4	92.7	37.8	4.3 -75.0	75.6	71.3
Larger models 32- 34 B params												
GPT-4	73.8	98	24.4	45.1 -28.7	98.0	53.7	34.8 -39	99	64.6	6.7 -67.1	95.0	89
CodeLlama-34b-Instruct-hf	50.0	86.0	36.0	29.3 -20.7	78.7	49.4	26.8 -23.2	79.3	52.4	4.9 -45.1	70.1	65.2
deepseek-coder-33b-instruct	71.3	95.7	24.4	48.8 -22.5	91.5	42.7	44.5 -26.8	86.0	41.5	9.1 -61.6	76.8	67.7
Qwen2.5-32B-Instruct	86.0	97.0	11.0	61.1 -24.9	92.7	31.1	56.1 -29.9	94.5	38.4	8.5 -77.5	81.7	73.2

Table 3: Evaluation results on the MBPP benchmark.

2*Model	Original			Incomplete			Ambiguous			Contradictory		
	Pass@1	SER	RIR	Pass@1	SER	RIR	Pass@1	SER	RIR	Pass@1	SER	RIR
Smaller models (6-7 B params)												
CodeLlama-7B-Instruct-hf	31.9	69.7	37.8	19.6 -12.3	55.5	35.9	20.3 -11.6	55.2	34.9	13.6 -18.3	67.0	53.5
deepseek-coder-6.7B-instruct	43.8	77.6	33.8	27.8 -16	63.8	36.4	28.0 -15.8	63.8	35.7	26.7 -17.1	72.0	45.3
Qwen2.5-7B-Instruct	46.8	78	31.2	29.4 -17.4	62.7	33.4	30.2 -16.6	62.5	32.3	14.2 -32.6	72	58.1
Larger models (32- 34 B params)												
GPT-4	38.6	93	54.6	10.4 -28.2	98.0	88.4	15.6 -23	95	80.3	8.4 -30.2	73	64.9
CodeLlama-34b-Instruct-hf	37.1	72.2	35.1	21.7 -15.4	54.5	32.9	22.6 -14.5	55	32.4	17.7 -20.0	69.1	51.4
deepseek-coder-33b-instruct	46.5	77.7	31.2	30.3 -16.2	64.7	34.4	29.9 -16.6	64.2	34.3	23.7 -22.8	71.9	11
Qwen2.5-32B-Instruct	50.3	78.3	28	31.2 -19.1	64.4	33.2	32.8 -17.5	64.5	31.7	14.7 -35.6	73.2	58.5

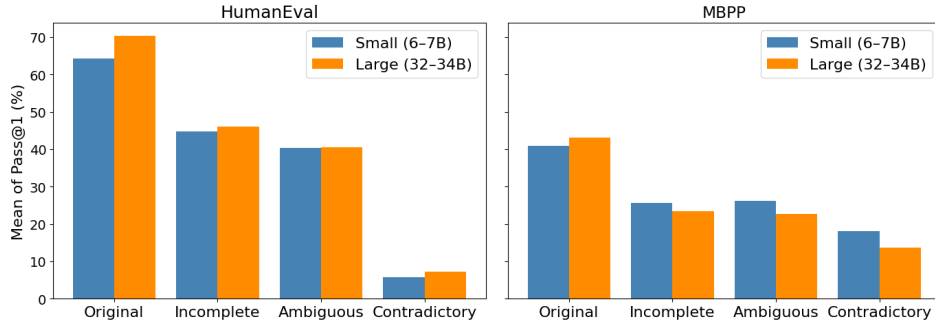


Figure 5: Mean Pass@1 accuracy (%) of small (6–7B) versus large (32–34B) model variants on HumanEval (left) and MBPP (right) across four task description categories: Original, Incomplete, Ambiguous, and Contradictory.

6.3 RQ3 – Error Type Analysis

To better understand the nature of model failures under unclear requirements, we analyze not only whether a generated solution runs successfully, but also whether it produces the correct logic. These findings demonstrate that unclear requirements not only degrade task performance but also alter the nature of failure modes [12]. Although all models maintained a high Successful Execution Rate (SER) across task descriptions conditions, this metric alone is insufficient. As shown in Figure 6, the Runnable but Incorrect Rate (RIR) increases significantly as description clarity declines.

This means models often produce syntactically valid code that compiles and executes, yet fails the functional requirements of the task, suggesting deep semantic misunderstandings. For instance, under contradictory task descriptions, RIR exceeds 80% for GPT-4 and approaches similar levels for other large models. In the case of ambiguous task descriptions, RIR remains high across all LLMs, with some small models such as Qwen-7B reaching over 70%. These results reinforce that execution alone is not a reliable proxy for correctness, especially when task descriptions contain subtle or conflicting defects. To further unpack these failures, we analyzed

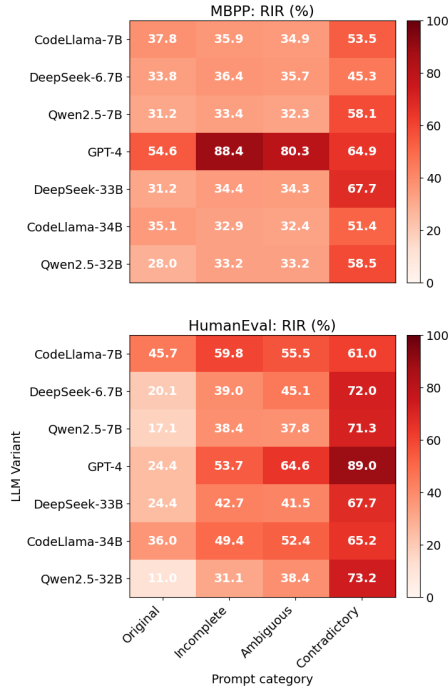


Figure 6: Runnable but incorrect rates (%) of seven LLM variants across four task description categories on MBPP (left) and HumanEval (right). Higher values indicate a greater share of executions that run but produce incorrect outputs.

the types of exceptions thrown by incorrect generations using our largest LLMs. We extract 3114 errors and categorize them in Table 4, showing the distribution of common Python exception types across each description category. Our findings reveal that:

- **Incomplete task descriptions:** are particularly prone to fundamental runtime failures [4] such as `NotFoundError` (82.4%), `TypeError` (37.3%), and `SyntaxError` (41.3%). These reflect structural gaps in the task description, often leading models to hallucinate or mishandle variables, arguments, or input types.
- **Ambiguous task descriptions:** in contrast, tend to generate semantically flawed logic that nonetheless executes. These are marked by elevated `AttributeError` (31.4%) and `KeyError` (38.9%), which typically emerge from misinterpreting vague specifications or assuming incorrect object structures.
- **Contradictory task descriptions** result in a mixed profile of errors, with moderate rates of logical faults across most categories, but notably higher `NameError` and `ValueError` rates. This suggests models are confused by internal inconsistencies, leading to conflicting variable usage or return semantics.
- **Original task descriptions**, though well formed, still yield a notable share of `NameError` (29.8%) and `ValueError` (30.1%). Interestingly, these are less common in ambiguous task descriptions, possibly because detailed instructions constrain

the model’s solution path, making small mistakes more impactful, while ambiguous task descriptions allow greater structural flexibility.

These findings demonstrate that unclear requirements not only degrade task performance but also alter the nature of failure modes. Incomplete task descriptions are more likely to trigger structural issues such as syntax and type errors; ambiguous task descriptions often result in semantically misaligned outputs; and contradictory task descriptions tend to produce logically inconsistent or invalid solutions. This highlights the importance of advancing LLMs’ ability to reason under unclear instructions, a critical step for safe and effective integration into real-world software development workflows where requirement clarity cannot be guaranteed.

RQ3 Results. Unclear task descriptions not only lower overall performance but also shift the nature of model failures. Incomplete descriptions lead to structural errors (e.g., `SyntaxError`, `TypeError`), ambiguous descriptions cause semantically incorrect but executable code (e.g., `AttributeError`, `KeyError`), and contradictory task descriptions produce logically inconsistent behavior (e.g., `NameError`, `ValueError`). These distinct error patterns highlight the need for input-sensitive debugging and adaptive mitigation strategies in LLM-powered development workflows. We also observed cases where a single generated snippet exhibited multiple issues at once, for example, missing parameters (incompleteness) combined with incorrect variable usage (semantic inconsistency). These compounded failures suggest that unclear descriptions can trigger layered reasoning problems rather than isolated mistakes. Quantifying how often such overlaps occur, as well as exploring whether mechanisms such as self-consistency checks or simple verification steps could mitigate them, represents a promising direction for future work.

7 Threats to validity

External Validity: Our findings are derived from Python code generation tasks, which may limit the generalizability of our conclusions. We performed our experiments on two popular benchmarks [21] (HumanEval and MBPP), focusing on relatively small, self-contained functions. These tasks, while varied in topic and complexity, do not cover other programming languages [40], multi-module projects, or domain-specific coding scenarios. In more complex real-world settings, LLMs may perform worse due to the increased complexity and the higher number of opportunities for task description defects.

Additionally, the description flaws we introduced (ambiguity, incompleteness, and contradiction) represent three common requirement issues, but not the full spectrum of possible imperfections (e.g., overly verbose, noisy, or poorly structured descriptions).

Another factor that may influence the results is the original quality of the HumanEval and MBPP tasks themselves. Prior work has shown that several tasks in these benchmarks already contain minor ambiguities or underspecified elements. As a result, part of the performance degradation we observe may stem from these pre-existing imperfections rather than from our mutations alone. Our injections amplify and control specific defect types, but the

Table 4: Distribution of error types across description categories. Rows (1–4) denote each category’s share of that error type, with the highest in each column highlighted in bold. The “Total” row reports the overall count of errors for each type.

Description Category	Attribute Error	Indentation Error	Index Error	Key Error	NotFound Error	Name Error	Syntax Error	Type Error	Value Error
Ambiguous	31.4%	28.6%	28.2%	38.9%	17.6%	26.7%	25.4%	29.4%	21.4%
Contradictory	21.6%	3.4%	17.9%	22.2%	0.0%	22.9%	22.2%	11.5%	20.4%
Incomplete	33.3%	52.1%	30.8%	16.7%	82.4%	20.6%	41.3%	37.3%	28.2%
Original	13.7%	16.0%	23.1%	22.2%	0.0%	29.8%	11.1%	21.7%	30.1%
Total	51	119	39	18	17	131	63	2573	103

baseline tasks are not entirely free of requirement issues, which should be considered when interpreting our results.

We partly addressed generalizability by confirming that robustness trends held across two distinct benchmarks and observing consistent patterns across multiple model families. We evaluated seven state-of-the-art code models, including GPT-4, DeepSeek Coder, CodeLlama, and Qwen in both small and large variants (6–7B and 32–34B). This diversity of models and tasks increases confidence that our conclusions are not tied to a single architecture or dataset.

Internal validity : The methodology we used for generating flawed task descriptions may constitute a threat to the internal validity. We created ambiguous, incomplete, and contradictory task descriptions by prompting GPT-4 to mutate the original benchmarks. This reliance on GPT-4 could introduce artifacts or systematic biases in the mutated task descriptions. We mitigated this risk via rigorous manual verification; multiple reviewers manually inspected and corrected (where needed) each generated task description to ensure it faithfully represented the intended ambiguity/inconsistency and retained fidelity to the original task.

Another potential confounding factor is the influence of model randomness on its outcomes. We used a consistent generation setting (greedy decoding) to limit nondeterminism, but if different sampling parameters were used, the success rates might vary. Another potential confounding factor is the influence of model randomness on its outcomes. We used a consistent generation setting (greedy decoding) to limit nondeterminism, but if different sampling parameters were used, the success rates might vary.

Another possible threat concerns the potential data contamination: some original tasks may have been seen in model training data, inflating baseline performance. As prior work shows, even small wording changes can cause sharp drops for memorized solutions [41]. Our mutations may break exact overlaps with training data, meaning that part of the observed degradation could stem from reduced familiarity rather than purely from reasoning limitations.

Finally, an important limitation is that we enforced ambiguity, contradiction, and incompleteness as disjoint categories to enable controlled comparisons. In real requirements engineering practice, however, these issues frequently co-occur: a single requirement may be simultaneously incomplete and ambiguous, or inconsistent and incomplete. Moreover, inconsistencies can arise not only from explicit logical contradictions but also from subtler sources such as terminology mismatches or semantic drift, as widely discussed in

requirements engineering literature [3, 25]. Our controlled mutations do not fully capture these natural overlaps, and the resulting categories may therefore be viewed as idealized abstractions. Future work should explore multi-label and overlapping issue types to better reflect real-world requirement defects.

8 Conclusion

As large language models continue to power code generation tools in real-world development environments, ensuring their robustness to unclear, user written task descriptions becomes increasingly critical. In this study, we conducted a systematic empirical analysis of LLM performance under flawed task descriptions by focusing on three prevalent issues: ambiguity, incompleteness, and contradiction.

Using controlled mutations of HumanEval and MBPP benchmarks, we showed that even minor task descriptions imperfections can lead to significant drops in Pass@1 accuracy and a rise in semantically incorrect but executable code (60-90% of the code that compiles is semantically incorrect). Although larger models generally outperform smaller ones in terms of compilation rates, the code they produce is (largely) semantically incorrect code. This means that scale alone does not guarantee robustness especially under contradictory or vague requirements.

Our analysis further reveals that different description flaws induce distinct failure patterns, from structural issues in incomplete task descriptions to semantic misalignment in ambiguous ones. These findings underscore the need for LLMs that are not only accurate but also resilient to the different kinds of unclear requirements frequently encountered in real world development [31]. Future research should investigate training strategies and diagnostic techniques that enable models to better interpret, detect, and recover from vague or under-specified task descriptions.

A promising avenue for future work lies in exploring self-consistency or self-checking mechanisms, where models verify or refine their own outputs. Such techniques may help LLMs recover from unclear requirements and mitigate some of the robustness limitations identified in this study.

Acknowledgments

This work is supported by the Luxembourg National Research Fund (FNR), Grant number C22/IS/17426831/MeMoRIA. Dr. Papadakis is supported by the Luxembourg National Research Fund (FNR) INTER/MOBILITY/2024/IS/18956086.

Table 5: Examples of mutated task descriptions for both HumanEval and MBPP datasets across the three description defect categories: Incomplete, Ambiguous, and Contradictory.

ID	Original Description	Incomplete	Ambiguous	Contradictory
MBPP/2	Write a function to find the similar elements from the given two tuple lists.	Write a function to find similar elements shared between lists.	Write a function to compare groups and find equal items.	Write a function to find similar elements from the given two tuple lists, but only include elements not in both.
MBPP/6	Write a python function to check whether the two numbers differ at one bit position only or not.	Write a function to check if two numbers differ.	Write a function that checks if two numerical values are distinguishable through a minimal binary alteration.	Write a python function to check whether two numbers differ at one bit position or two.
MBPP/10	Write a function to get the n smallest items from a dataset.	Write a function to get items from a dataset.	Write a function to select the smallest of a data group.	Write a function to get the n smallest items from a dataset, but return one item only.
MBPP/22	Write a function to find the first duplicate element in a given array of integers.	Write a function to find the first duplicate in an array.	Write a function to identify first repeating items in a group of numbers.	Write a function to find the first duplicate element in a given array.return last one.
MBPP/34	Write a python function to find the missing number in a sorted array.	Write a function to find a missing value in a list.	Write a function to look for gaps in a sequence.	Write a python function to find the missing number in a sorted array, but always return the first number.
HumanEval/13	def greatest_common_divisor(a: int, b: int) -> int: Return the greatest common divisor of two integers a and b >>> greatest_common_divisor(3, 5) 1 >>> greatest_common_divisor(25, 15) 5	Write a program that calculate the common divisor with two values	Write a Python instruction that finds the biggest whole number that imples divisibility exactly into two given amounts.	def greatest_common_divisor(a: str, b: str) -> str: Return a greatest common divisor of two integers a and b
HumanEval/16	def count_distinct_characters(string: str) -> int: """ Given a string, find out how many distinct characters (regardless of case) does it consist of >>> count_distinct_characters('xyzXYZ') 3 >>> count_distinct_characters('Jerry') 4 """	def count_distinct_characters(string):""" find out how many distinct characters does it consist of. """	Come up with a way to figure out how many different letters show up in a word, treating big and small versions of the same letter as the same thing.	def count_distinct_characters(string: str, ignore_case: bool) -> str: """ Given a string, find out how many distinct characters it consists. please return the number of similar characters. """
HumanEval/23	def strlen(string: str) -> int: """ Return length of given string >> strlen("") 0 >> strlen('abc') 3 """	def strlen: """ Return length of string """	Create a way to determine how much content is in the given piece of text.	def strlen(string: str) -> str: """ Return length of given string in string format without using the len function. >> strlen(11) 0 """
HumanEval/27	def flip_case(string: str) -> str: """ For a given string, flip lowercase characters to uppercase and uppercase to lowercase. >>> flip_case('Hello') 'hELLO' """	For a given input, flip lowercase characters to uppercase and uppercase to lowercase.	Given a string, transform each letter so that those that normally stand tall now crouch, and those that usually crouch stand tall.	Take a piece of text and transform it so that all letters that are normally written in a smaller form become capitalized, while also ensuring that no uppercase letters are altered in any way.
HumanEval/28	from typing import List def concatenate(strings: List[str]) -> str: """ Concatenate list of strings into a single string >>> concatenate([]) "" >>> concatenate(['a', 'b', 'c']) 'abc' """	Concatenate list of strings into a single string.	Given a bunch of pieces of text, figure out how to make them into one continuous bit.	from typing import List def concatenate(strings: List[str]) -> int: """ Concatenate list of strings into a single string but return an integer. The function should also work for lists of integers. """

References

- [1] 1998. IEEE Recommended Practice for Software Requirements Specifications. *IEEE Std 830-1998* (1998), 1–40. doi:10.1109/IEEESTD.1998.88286
- [2] DeepSeek AI. 2024. DeepSeek-Coder: Unlocking Code Generation Abilities with Large Language Models. *arXiv preprint arXiv:2401.14196* (2024). <https://arxiv.org/abs/2401.14196>
- [3] Raian Ali, Fabiano Dalpiaz, and Paolo Giorgini. 2013. Reasoning with contextual requirements: Detecting inconsistency and conflicts. *Information and Software Technology* 55, 1 (2013), 35–57.
- [4] Maalem Abdulsattar Atiyaa and Marwa Adeeb Al-Jawaherry. 2025. Exploring deep learning for accurate software cost estimation: A long short-term memory (LSTM) approach. In *AIP Conference Proceedings*, Vol. 3264. AIP Publishing LLC, 040019.
- [5] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. 2021. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732* (2021).
- [6] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. 2021. Program Synthesis with Large Language Models. *arXiv preprint arXiv:2108.07732* (2021).
- [7] Tianfeng Chai and Roland R Draxler. 2014. Root mean square error (RMSE) or mean absolute error (MAE)?—Arguments against avoiding RMSE in the literature. *Geoscientific model development* 7, 3 (2014), 1247–1250.
- [8] Haoyang Chen, Botong Xu, Louis Zhong Rui Wong, and Kaiyang Zhong. 2025. Enhancing software effort estimation through reinforcement learning-based project management-oriented feature selection. *International Journal of Managing Projects in Business* (2025).
- [9] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgren Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. Evaluating Large Language Models Trained on Code. (2021). [arXiv:2107.03374](https://arxiv.org/abs/2107.03374) [cs.LG]
- [10] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).
- [11] Arghavan Moradi Dakhel, Vahid Majdinasab, Amin Nikanjam, Foutse Khomh, Michel C Desmarais, and Zhen Ming Jack Jiang. 2023. Github copilot ai pair programmer: Asset or liability? *Journal of Systems and Software* 203 (2023), 111734.
- [12] Shihan Dou, Haoxiang Jia, Shenxi Wu, Huiyuan Zheng, Weikang Zhou, Muling Wu, Mingxu Chai, Jessica Fan, Caishuang Huang, Yunbo Tao, et al. 2024. What's wrong with your code generated by large language models? an extensive study. *arXiv preprint arXiv:2407.06153* (2024).
- [13] Shihan Dou, Haoxiang Jia, Shenxi Wu, Huiyuan Zheng, Weikang Zhou, Muling Wu, Mingxu Chai, Jessica Fan, Caishuang Huang, Yunbo Tao, Yan Liu, Enyu Zhou, Ming Zhang, Yuhao Zhou, Yueming Wu, Rui Zheng, Ming Wen, Rongxiang Weng, Jingang Wang, Xunliang Cai, Tao Gui, Xipeng Qiu, Qi Zhang, and Xuanjing Huang. 2024. What's Wrong with Your Code Generated by Large Language Models? An Extensive Study. *arXiv preprint arXiv:2407.06153* (2024).
- [14] Adithya V Ganesan, Yash Kumar Lal, August Håkan Nilsson, and H Andrew Schwartz. 2023. Systematic evaluation of GPT-3 for zero-shot personality estimation. *arXiv preprint arXiv:2306.01183* (2023).
- [15] Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, and Jacob Steinhardt. 2021. Measuring Coding Challenge Competence With APPS. *arXiv preprint arXiv:2105.09938* (2021). <https://arxiv.org/abs/2105.09938>
- [16] Sepp Hochreiter and Jürgen Schmidhuber. 1997. Long short-term memory. *Neural computation* 9, 8 (1997), 1735–1780.
- [17] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. 2024. Large Language Models for Software Engineering: A Systematic Literature Review. *ACM Trans. Softw. Eng. Methodol.* 33, 8, Article 220 (Dec. 2024), 79 pages. doi:10.1145/3695988
- [18] Anca-Elena Iordan. 2024. An optimized LSTM neural network for accurate estimation of software development effort. *Mathematics* 12, 2 (2024), 200.
- [19] Magne Jorgensen and Martin Shepperd. 2006. A systematic review of software development cost estimation studies. *IEEE Transactions on software engineering* 33, 1 (2006), 33–53.
- [20] Yuhang Lai, Chengxi Li, Yiming Wang, Tianyi Zhang, Ruiqi Zhong, Luke Zettlemoyer, Wen-tau Yih, Daniel Fried, Sida Wang, and Tao Yu. 2023. DS-1000: a natural and reliable benchmark for data science code generation. In *Proceedings of the 40th International Conference on Machine Learning (Honolulu, Hawaii, USA) (ICML '23)*. JMLR.org, Article 756, 27 pages.
- [21] Yasir Mahmood, Nazri Kama, Azri Azmi, Ahmad Salman Khan, and Mazlan Ali. 2022. Software effort estimation accuracy prediction of machine learning techniques: A systematic performance evaluation. *Software: Practice and experience* 52, 1 (2022), 39–65.
- [22] Ruchika Malhotra. 2015. A systematic review of machine learning techniques for software fault prediction. *Applied Soft Computing* 27 (2015), 504–518.
- [23] Bertrand Meyer. 1985. On Formalism in Specifications. *IEEE Softw.* 2, 1 (1985), 6–26. doi:10.1109/MS.1985.229776
- [24] Lloyd Montgomery, Davide Fucci, Abir Bouraffa, Lisa Scholz, and Walid Maalej. 2022. Empirical research on requirements quality: a systematic mapping study. *Requir. Eng.* 27, 2 (2022), 183–209. doi:10.1007/S00766-021-00367-Z
- [25] Bashar Nuseibeh, Steve Easterbrook, and Alessandra Russo. 2001. Making inconsistency respectable in software development. *Journal of systems and software* 58, 2 (2001), 171–180.
- [26] OpenAI. 2023. GPT-4 Technical Report. <https://cdn.openai.com/papers/gpt-4.pdf>.
- [27] Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. 2025. Asleep at the keyboard? assessing the security of github copilot's code contributions. *Commun. ACM* 68, 2 (2025), 96–105.
- [28] Zepeng Qin, Xiangyang Li, Zhongzhi Huang, et al. 2024. Qwen-Code: A Powerful Language Model for Code Understanding and Generation. *arXiv preprint arXiv:2403.05530* (2024). <https://arxiv.org/abs/2403.05530>
- [29] Mizanur Rahman, Teresa Gonçalves, and Hasan Sarwar. 2023. Review of existing datasets used for software effort estimation. *Int. J. Adv. Comput. Sci. Appl.* 14, 7 (2023).
- [30] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. 2018. Semantically equivalent adversarial rules for debugging NLP models. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (volume 1: long papers)*, 856–865.
- [31] Ritu and Pankaj Bhambri. 2025. Enhancing software development effort estimation with a cloud-based data framework using case points, fuzzy logic, and machine learning. *Discover Computing* 28, 1 (2025), 143.
- [32] Bruno Budel Rossi and Lisandra Manzoni Fontoura. [n. d.]. AI-Based Approaches for Software Tasks Effort Estimation: A Systematic Review of Methods and Trends. ([n. d.]).
- [33] Baptiste Roziere, Loubna Allal, Lewis Tunstall, et al. 2023. Code Llama: Open Foundation Models for Code. *arXiv preprint arXiv:2308.12950* (2023). <https://arxiv.org/abs/2308.12950>
- [34] Ángel J Sánchez-García, María Saarayim González-Hernández, Karen Cortés-Verdin, and Juan Carlos Pérez-Arriaga. 2024. Software Estimation in the Design Stage with Statistical Models and Machine Learning: An Empirical Study. *Mathematics* 12, 7 (2024), 1058.
- [35] Md Mahfuzul Islam Shamim, Abu Bakar bin Abdul Hamid, Tadiwa Elisha Nyamasvisva, and Najmus Saqib Bin Rafi. 2025. Advancement of Artificial Intelligence in Cost Estimation for Project Management Success: A Systematic Review of Machine Learning, Deep Learning, Regression, and Hybrid Models. *Modelling* 6, 2 (2025), 35.
- [36] Wilamis KN Silva, Bernan R Nascimento, Pérciles Miranda, and Emanuel P Vicente. [n. d.]. Predictive Regression Models of Machine Learning for Effort Estimation in Software Teams: An Experimental Study. ([n. d.]).
- [37] Victor Uc-Cetina. 2023. Recent advances in software effort estimation using machine learning. *arXiv preprint arXiv:2303.03482* (2023).
- [38] Karthik Valmeekam, Alberto Olmo, Sarath Sreedharan, and Subbarao Kambhampati. 2022. Large language models still can't plan (a benchmark for LLMs on planning and reasoning about change). In *NeurIPS 2022 Foundation Models for Decision Making Workshop*.
- [39] Axel van Lamsweerde. 2009. *Requirements Engineering - From System Goals to UML Models to Software Specifications*. Wiley. <http://eu.wiley.com/WileyCDA/WileyTitle/productCd-EHEP000863.html>
- [40] Jiexin Wang, Xitong Luo, Liuwen Cao, Hongkui He, Hailin Huang, Jiayuan Xie, Adam Jatowt, and Yi Cai. 2024. Is your ai-generated code really safe? evaluating large language models on secure code generation with codeseeval. *arXiv preprint arXiv:2407.02395* (2024).
- [41] Shiqi Wang, Zheng Li, Haifeng Qian, Chenghao Yang, Zijian Wang, Mingyue Shang, Varun Kumar, Samson Tan, Baishakhi Ray, Parminder Bhatia, Ramesh Nallapati, Murali Krishna Ramanathan, Dan Roth, and Bing Xiang. 2022. ReCode: Robustness Evaluation of Code Generation Models. *arXiv preprint arXiv:2212.10264* (2022).
- [42] Fangzhou Wu, Xiaogeng Liu, and Chaowei Xiao. 2023. Deceptprompt: Exploiting llm-driven code generation via adversarial natural language instructions. *arXiv preprint arXiv:2312.04730* (2023).
- [43] Jie JW Wu and Fatemeh H Fard. 2025. Humanevalcomm: Benchmarking the communication competence of code generation for llms and llm agents. *ACM*

- Transactions on Software Engineering and Methodology* 34, 7 (2025), 1–42.
- [44] Burak Yetistiren, Isik Ozsoy, and Eray Tuzun. 2022. Assessing the quality of GitHub copilot's code generation. In *Proceedings of the 18th international conference on predictive models and data analytics in software engineering*. 62–71.
 - [45] Zihan Zheng, Shahad Hardan, Darya Taratynova, Abdelmajid Essofi, Karthik Nandakumar, and Mohammad Yaqub. 2025. LiveCodeBench Pro: How Do Olympiad Medalists Judge LLMs in Competitive Programming? *arXiv preprint arXiv:2506.11928* (2025). <https://arxiv.org/abs/2506.11928>
 - [46] Zibin Zheng, Kaiwen Ning, Qingyuan Zhong, Jiachi Chen, Wenqing Chen, Lianghong Guo, Weicheng Wang, and Yanlin Wang. 2025. Towards an understanding of large language models in software engineering tasks. *Empirical Software Engineering* 30, 2 (2025), 50.
 - [47] Kaijie Zhu, Qinlin Zhao, Hao Chen, Jindong Wang, and Xing Xie. 2024. Prompt-bench: A unified library for evaluation of large language models. *Journal of Machine Learning Research* 25, 254 (2024), 1–22.